



INDUS11

AI Financial Risk & Fraud Decision Engine

Project overview · idea, technology, and plan

Joshi Om · Krish Gajera · Drashti Dedaniya

Semester Group Project · Algoma University

Real-time fraud decisions, explained in plain English

The domain

Financial fraud detection. Banks must approve or block each payment in milliseconds — rule-only systems miss new fraud tricks, and pure AI models can't explain why they blocked someone.

Our idea

- Score every transaction with three independent engines — business rules, graph analysis of fraud rings, and a RAG-grounded LLM
- Combine them into one 0–100 score with clear thresholds
- Return the decision plus a natural-language explanation an analyst can act on



APPROVE score 0–39

Clean payment — release instantly



REVIEW score 40–69

Suspicious — queue for an analyst



BLOCK score 70–100

High risk — stop the transaction

Existing systems — and our niche

What's already out there

FICO Falcon

Neural networks + shared bank data; scores in milliseconds

Feedzai

Real-time AI fraud + anti-money-laundering platform

Featurespace ARIC

Adaptive behavioural profile built per customer

NICE Actimize

Enterprise fraud + AML investigation suite

Amazon Fraud Detector · Stripe Radar

Managed cloud ML fraud services



Where Indus11 fits

- Explains every decision in plain English — not a black box
- Rules + graph + a capped LLM — every point is traceable
- Built-in fraud-ring detection via a graph database
- Open-source · self-hostable · runs free on a laptop

A working fraud-detection API and dashboard



Real-time analysis API

POST /analyze runs a transaction through all engines and returns a decision; 12 REST endpoints in total.



Live dashboard

React + Recharts: risk distribution, score histogram, and a feed of flagged transactions.



Synthetic test dataset

500 accounts and ~1,000 transactions with 3 fraud rings planted to test detection.



Explained decisions

Every result returns APPROVE / REVIEW / BLOCK with a plain-English reason.

Scope: a working prototype on synthetic data that demonstrates the detection approach end-to-end — not a production banking deployment.

Open-source stack, runs on one laptop



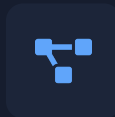
FastAPI

Async Python 3.12 API — the gateway that runs all engines



MongoDB

Transactions & accounts (Beanie ODM) — flexible document store



Neo4j

Graph database — fraud-ring & shared-device detection



Redis

Profile cache + rolling velocity windows for speed



LangChain + Ollama

RAG pipeline with a local LLM (llama3) — free & offline



ChromaDB

Vector store — 58 fraud-pattern documents for retrieval



React + Vite

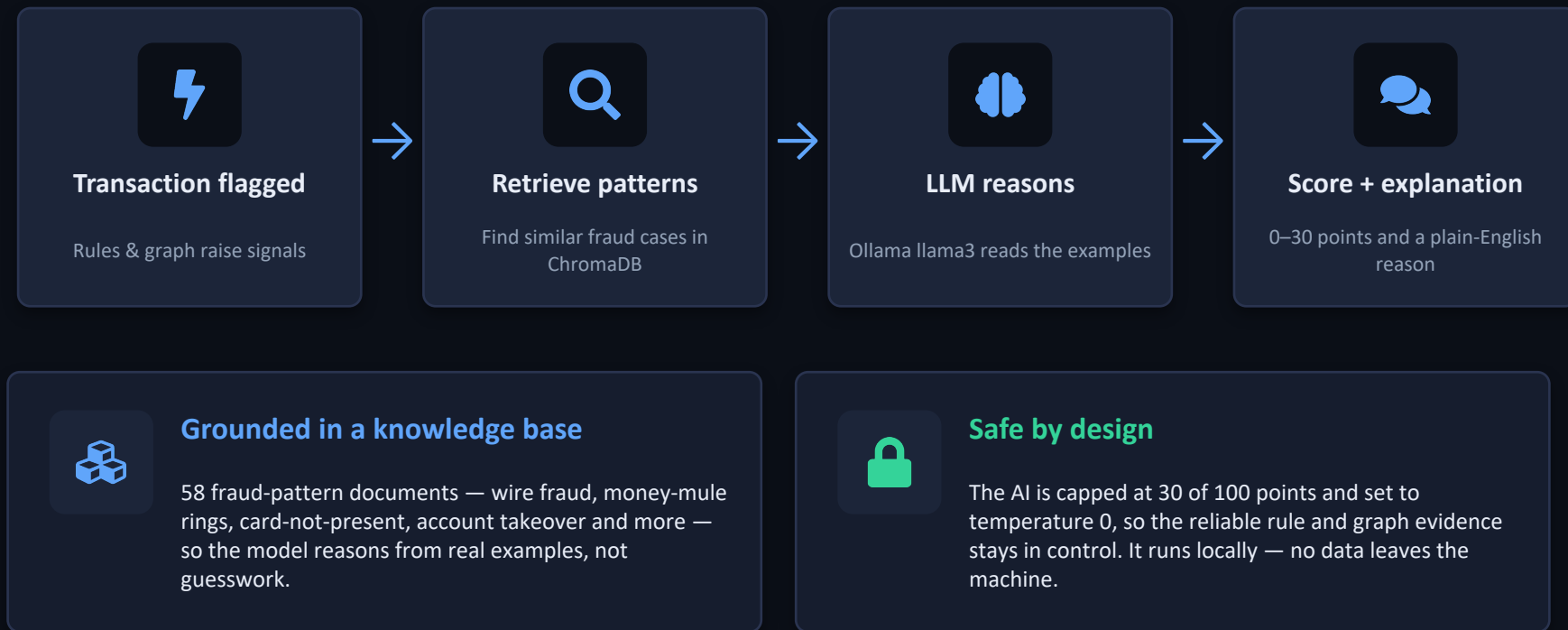
Live operations dashboard with Recharts



Docker Compose

One-command spin-up of the whole stack

How the AI layer scores and explains



Security & AI — in plain terms



Cybersecurity

A security guard for money

- Catches stolen accounts — many accounts sharing one device or IP
- Catches card testing — too many payments too fast
- Finds fraud rings — hidden money-laundering groups
- Protects itself — input checks, rate limits, full audit log



AI & Machine Learning

Recognises patterns, explains itself

- ML finds transactions similar to known fraud — like a photo app grouping faces
- AI writes a plain-English reason for every decision
- Runs privately and offline — no data leaves the machine
- Kept on a short leash — capped so the hard checks stay in control

Grounded in the research



APATE — network-based fraud detection (Van Vlasselaer et al., 2015)

Combining graph (network) features with traditional features reaches AUC > 0.98 and decides within the industry's 6-second limit.

For us: Evidence that rules + graph together beat either alone — our core design.



Explainable AI for fraud — SHAP-based studies (2024–2025)

Explainability improves analyst trust and regulatory compliance, especially under heavy class imbalance.

For us: Backs our explanation-first approach — a reason with every decision.



LLM-enhanced graph fraud detection (2025)

The latest research pairs LLMs with graphs for fraud — while flagging LLM security as an open risk.

For us: Our RAG + graph design sits on this frontier — and is why we cap the LLM at 30/100.

Takeaway: the literature backs our two core bets — graph + traditional features beat either alone, and explainability materially improves fraud systems.

Contributions — clear ownership per layer



Joshi Om

Member A

Platform & Rules

- FastAPI server & REST API design
- MongoDB schema · Beanie ODM
- Redis caching & velocity windows
- Rule engine — amount, velocity, blacklist



Krish Gajera

Member B

Graph Intelligence

- Neo4j schema & Cypher fraud queries
- Fraud-ring & shared-device detection
- Synthetic fraud dataset (500 accounts)
- Fraud-label propagation



Drashti Dedaniya

Member C

AI & Experience

- RAG pipeline — LangChain + ChromaDB
- LLM prompt & local Ollama integration
- Decision engine — score aggregation
- React dashboard & stats API

References & sources

Research papers

1. APATE: automated credit-card fraud detection using network-based extensions

Van Vlasselaer et al. (2015) · Decision Support Systems · ScienceDirect: S0167923615000846

2. A graph-based, semi-supervised credit-card fraud detection system

Lebichot et al. (2017) · Springer · doi:10.1007/978-3-319-50901-3_57

3. On the potential of network-based features for fraud detection

(2024) · arXiv:2402.09495

4. Explainable-AI-enhanced ensemble learning for financial fraud detection

Vijayanand & Smrithy (2025) · SAGE · doi:10.1177/18724981241289751

5. Security & privacy challenges of large language models: a survey

(2024) · arXiv:2402.00888

Industry sources

FICO Falcon · Feedzai · Featurespace ARIC · NICE Actimize — vendor & industry overviews (fico.com, intellisoft.io).